TALLER PRÁCTICO FORMATIVO: ARQUITECTURA WEB CLIENTE Y MANIPULACIÓN AVANZADA DEL DOM

Arquitectura Modular de Componentes en JavaScript Puro y Consumo Asíncrono con Promesas

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS INTELIGENTES

INFORMACIÓN DE LA ACTIVIDAD — IDENTIFICACIÓN

CÓDIGO:	09810 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
NRC / GRUPO:	12378 / 12377 (Grupos 001 y 003)	**CARÁCTER:**	Taller Práctico Formativo Guiado (Sin Calificación Sumativa)
MODALIDAD:	Equipos de 2 Estudiantes	**DURACIÓN:**	2 Horas en Clase + Cierre Autónomo
POLÍTICA IAG:	Nivel 2: Asistencia de Consulta y Depuración	**PONDERACIÓN:**	Formativo (Alineación RAA 3 / Preparación Parcial 3)
ESTUDIANTE 1:	_____	**CÓDIGO:**	_____
ESTUDIANTE 2:	_____	**CÓDIGO:**	_____

1 Contexto: El Paradigma de Componentes en Vanilla JavaScript

En las arquitecturas modernas de aplicaciones web de una sola página (*Single Page Applications* - *SPAs*), el documento HTML inicial ya no contiene estructuras visuales fijas ni elementos estáticos como encabezados, formularios o tablas. En su lugar, el archivo `index.html` actúa exclusivamente como un **contenedor raíz vacío** (`<div id="root"></div>`) acompañado de la importación del script principal de JavaScript.

Toda la interfaz gráfica, la actualización dinámica y la manipulación del árbol DOM se construyen en tiempo de ejecución mediante código JavaScript puro. Para que este código sea mantenible, escalable y profesional, adoptamos el **Paradigma de Componentes Funcionales**:

- **¿Qué es un Componente en Vanilla JS?:** Es una función modular reutilizable que recibe parámetros (conocidos como **props**) y retorna un nodo del DOM completamente configurado (con sus clases, atributos, eventos y contenido de texto seguro).
- **Principios de la Arquitectura Modular de Componentes:**
  - **Props hacia abajo:** Los componentes padres pasan datos de configuración y estado hacia los componentes hijos mediante argumentos de función.
  - **Event Callbacks hacia arriba:** Los componentes hijos notifican acciones del usuario (como presionar "Eliminar" o enviar el formulario) invocando funciones callback pasadas en sus props (ej. `onAddTask`, `onDeleteTask`).
  - **Estado Centralizado (State):** El componente raíz (`App`) mantiene la única fuente de verdad de la aplicación (el array de tareas, el estado de carga y los mensajes de error). Cuando el estado cambia, se dispara una actualización dinámica de la interfaz.
- **Consumo Asíncrono con Promesas (Service Layer):** El acceso a la red se encapsula en un módulo de servicio independiente

(`taskService.js`). Cada operación HTTP (`GET`, `POST`, `DELETE`) retorna una **Promesa** que transita de forma determinística por sus tres estados:

a) **Pending:** La petición está en vuelo; el componente `App` renderiza el componente `LoadingSpinner`.

b) **Fulfilled:** La Promesa se resuelve con éxito; se actualiza la lista de tareas en el estado y se dibuja el componente `TaskList`.

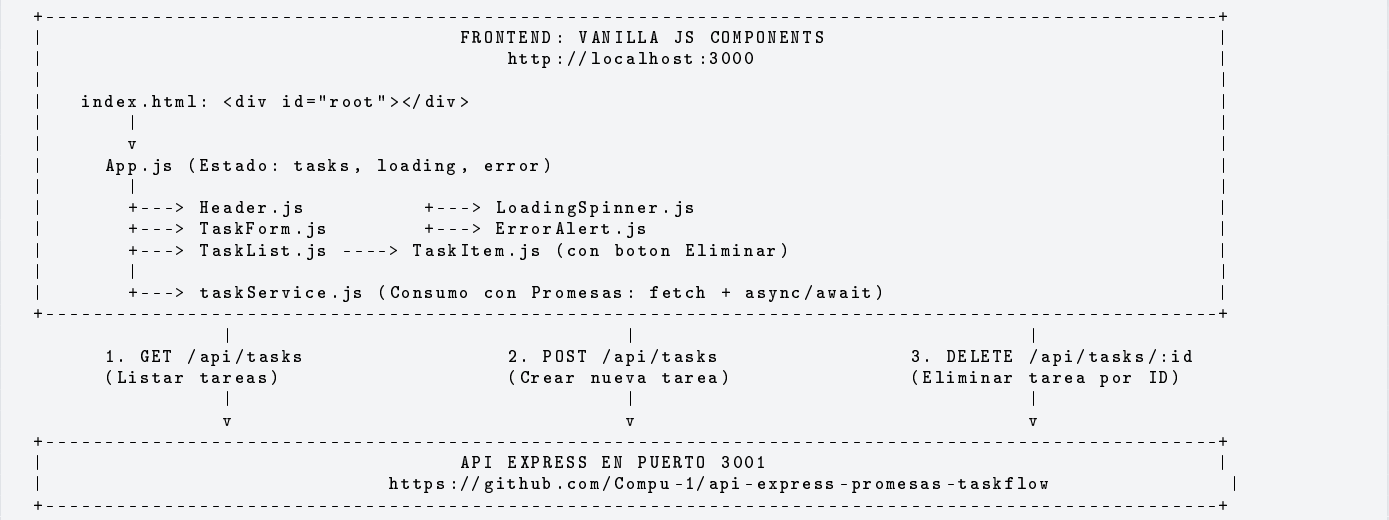
c) **Rejected:** La red o el servidor fallaron; se captura el motivo del rechazo en el bloque `catch` y se renderiza el componente `ErrorAlert`.

Propósito Formativo del Taller

Aprender a construir una aplicación web completa utilizando **Arquitectura de Componentes en Vanilla JS** y manipulación programática del DOM, partiendo de un HTML con una sola etiqueta (`<div id="root"></div>`). Desarrollaremos una suite modular de componentes (Header, TaskForm, TaskItem, TaskList, LoadingSpinner, ErrorAlert y App) y nos conectaremos a un servidor Express sencillo que implementa operaciones `GET`, `POST` y `DELETE` mediante **Promesas** con `fetch` y `async / await`.

2 La API de Soporte: Servidor Express como Caja Negra

Para abastecer nuestra aplicación frontend con datos reales, utilizaremos el servidor Express provisto por la cátedra. En este curso **no programamos el backend**; la API actúa como una caja negra en el puerto `3001` que maneja una lista de tareas en memoria.



2.1 Código Completo del Servidor (server.js)

A continuación se presenta el código del backend para su lectura y contexto:

```
import express from 'express';
import cors from 'cors';

const app = express();
const PORT = 3001;

app.use(cors());           // Habilita peticiones cruzadas desde el frontend en localhost:3000
app.use(express.json());   // Middleware para interpretar cuerpos JSON en POST
```

```
// Lista de tareas inicial en memoria
let tareas = [
  { id: 1, title: "Comprender la estructura basica de HTML y CSS", completed: true },
  { id: 2, title: "Aprender a seleccionar y modificar elementos del DOM", completed: true },
  { id: 3, title: "Entender que es una Promesa y sus tres estados en JS", completed: false },
  { id: 4, title: "Consumir una API externa usando fetch y async/await", completed: false },
  { id: 5, title: "Renderizar dinamicamente los datos en la interfaz web", completed: false }
];

// 1. GET /api/tasks: Obtener todas las tareas
app.get('/api/tasks', (req, res) => res.json(tareas));

// 2. POST /api/tasks: Registrar nueva tarea
app.post('/api/tasks', (req, res) => {
  const { title } = req.body;
  if (!title || !title.trim()) return res.status(400).json({ error: "Titulo requerido" });
  const nuevaTarea = { id: Date.now(), title: title.trim(), completed: false };
  tareas.push(nuevaTarea);
  res.status(201).json(nuevaTarea);
});

// 3. DELETE /api/tasks/:id: Eliminar tarea por ID
app.delete('/api/tasks/:id', (req, res) => {
  const id = req.params.id;
  tareas = tareas.filter((t) => String(t.id) !== String(id));
  res.json({ message: "Tarea eliminada exitosamente", id });
});

app.listen(PORT, () => console.log(`Servidor activo en http://localhost:${PORT}`));
```

3 Procedimiento Experimental Paso a Paso: Construyendo el Front

A continuación desarrollaremos la aplicación web estructurada modularmente en componentes funcionales.

3.1 Fase 1: Puesta en Marcha de la API de Soporte (5 Minutos)

En una terminal, clone e inicie el servidor de la cátedra:

```
git clone https://github.com/Compu-1/api-express-promesas-taskflow.git
cd api-express-promesas-taskflow
npm install && npm start
```

Verifique en su navegador que `http://localhost:3001/api/tasks` responde con el JSON de tareas. Mantenga esta terminal abierta.

3.2 Fase 2: Estructura del Proyecto y el HTML Mínimo (10 Minutos)

Abra una **segunda terminal** en una ubicación independiente para crear el proyecto cliente:

```
mkdir taskflow-components && cd taskflow-components
mkdir -p css js/services js/components
touch index.html css/styles.css js/main.js js/services/taskService.js
touch js/components/Header.js js/components/TaskForm.js js/components/TaskItem.js
touch js/components/TaskList.js js/components/LoadingSpinner.js js/components/ErrorAlert.js js/components/App.js
```

Edite `index.html`. Observe que el cuerpo del documento **solo tiene la etiqueta raíz** `<div id="root"></div>` y el enlace al script:

```
<!DOCTYPE html>
<html lang="es">
<head>
```

```
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TaskFlow - Arquitectura de Componentes en Vanilla JS</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
  <!-- Contenedor Raiz Unico: Toda la UI se crea dinamicamente via JS -->
  <div id="root"></div>

  <!-- Punto de entrada de la aplicacion -->
  <script type="module" src="js/main.js"></script>
</body>
</html>
```

Agregue los estilos visuales en `css/styles.css`:

```
/* css/styles.css: Paleta moderna institucional y estados de componentes */
:root { --primario: #5454E9; --exito: #4CB978; --error: #EF4444; --fondo: #F3F4F6; }
body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; background: var(--fondo);
    display: flex; justify-content: center; padding: 2.5rem 1rem; margin: 0; }
#root { width: 100%; max-width: 560px; }
.app-container { background: white; padding: 2.2rem; border-radius: 12px; box-shadow: 0 4px 18px rgba(0,0,0,0.08);
    }
.app-header h1 { margin: 0 0 0.3rem 0; color: #1F2937; font-size: 1.55rem; }
.app-subtitle { margin: 0; color: #6B7280; font-size: 0.88rem; }
.task-form { display: flex; gap: 0.5rem; margin: 1.5rem 0 1rem 0; }
.task-form input { flex: 1; padding: 0.7rem 0.9rem; border: 1.5px solid #D1D5DB; border-radius: 8px; font-size: 0.9
    rem; }
.task-form input:focus { outline: none; border-color: var(--primario); }
.btn-add { background: var(--exito); color: white; border: none; padding: 0.7rem 1.2rem; border-radius: 8px; font-
    weight: 600; cursor: pointer; white-space: nowrap; }
.btn-add:hover { background: #3EA668; }
.btn-refresh { background: var(--primario); color: white; border: none; padding: 0.65rem 1.2rem; border-radius: 8px
    ; font-weight: 600; font-size: 0.9rem; cursor: pointer; width: 100%; margin-bottom: 1rem; }
.btn-refresh:hover { background: #3E3EC8; }
.btn-add:disabled, .btn-refresh:disabled { background: #9CA3AF; cursor: not-allowed; }
.loading-container { display: flex; align-items: center; justify-content: center; gap: 0.6rem; padding: 0.8rem 0;
    color: #4B5563; font-size: 0.88rem; }
.spinner { width: 18px; height: 18px; border: 2.5px solid #E5E7EB; border-top-color: var(--primario); border-radius
    : 50%; animation: girar 0.8s linear infinite; }
@keyframes girar { to { transform: rotate(360deg); } }
.error-alert { background: #FEE2E2; color: #991B1B; padding: 0.85rem; border-radius: 8px; font-size: 0.88rem;
    border: 1px solid #FCA5A5; margin-bottom: 1rem; display: flex; justify-content: space-between; align-items:
    center; gap: 0.5rem; }
.btn-retry { background: #DC2626; color: white; border: none; padding: 0.3rem 0.7rem; border-radius: 6px; font-size
    : 0.8rem; cursor: pointer; }
.task-list { list-style: none; padding: 0; margin: 0; display: flex; flex-direction: column; gap: 0.5rem; }
.task-item { display: flex; align-items: center; justify-content: space-between; padding: 0.75rem 0.9rem;
    background: #F9FAFB; border: 1px solid #E5E7EB; border-radius: 8px; gap: 0.8rem; }
.task-title { flex: 1; color: #1F2937; }
.task-item.completed .task-title { text-decoration: line-through; color: #9CA3AF; }
.task-actions { display: flex; align-items: center; gap: 0.5rem; }
.badge { font-size: 0.72rem; padding: 0.2rem 0.45rem; border-radius: 12px; font-weight: 600; white-space: nowrap; }
.badge-completed { background: #D1FAE5; color: #065F46; }
.badge-pending { background: #FEF3C7; color: #92400E; }
.btn-delete { background: transparent; border: 1px solid #FCA5A5; color: var(--error); border-radius: 6px; padding:
     0.25rem 0.55rem; font-size: 0.76rem; cursor: pointer; font-weight: 600; }
.btn-delete:hover { background: #FEE2E2; }
.empty-message { text-align: center; color: #9CA3AF; padding: 1.5rem 0; }
```

3.3 Fase 3: Módulo de Servicio con Promesas (js/services/taskService.js) (15 Minutos)

Encapsulamos las llamadas `fetch` que retornan Promesas en un módulo reutilizable:

```
// js/services/taskService.js: Capa de comunicacion asincrona con la API
const API_URL = 'http://localhost:3001/api/tasks';

/**
 * 1. GET: Retorna una Promesa que resuelve en el array de tareas
 */
export async function getTasks() {
  const response = await fetch(API_URL);
  if (!response.ok) {
    throw new Error(`Fallo al consultar tareas. Codigo HTTP: ${response.status}`);
  }
  return await response.json(); // Segunda Promesa: deserializa JSON
}

/**
 * 2. POST: Retorna una Promesa que resuelve en el nuevo objeto tarea
 */
export async function createTask(title) {
  const response = await fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: title })
  });
  if (!response.ok) {
    throw new Error(`Fallo al crear la tarea. Codigo HTTP: ${response.status}`);
  }
  return await response.json();
}

/**
 * 3. DELETE: Retorna una Promesa que confirma la eliminacion
 */
export async function deleteTask(id) {
  const response = await fetch(`${API_URL}/${id}`, {
    method: 'DELETE'
  });
  if (!response.ok) {
    throw new Error(`Fallo al eliminar la tarea. Codigo HTTP: ${response.status}`);
  }
  return await response.json();
}
```

3.4 Fase 4: Componentes Funcionales de Presentación (30 Minutos)

Construiremos cada componente como una función pura que recibe `props` y retorna un nodo del DOM manipulado de forma segura con `createElement` y `textContent`.

3.4.1. Componente 1: Header (js/components/Header.js)

```
export function Header({ title, subtitle }) {
  const header = document.createElement('header');
  header.className = 'app-header';

  const h1 = document.createElement('h1');
  h1.textContent = title;

  const p = document.createElement('p');
```

```
  p.className = 'app-subtitle';
  p.textContent = subtitle;

  header.append(h1, p);
  return header;
}
```

3.4.2. Componente 2: TaskForm (js/components/TaskForm.js)

Emite un evento callback `onAddTask` al enviar el formulario:

```
export function TaskForm({ onAddTask, disabled }) {
  const form = document.createElement('form');
  form.className = 'task-form';

  const input = document.createElement('input');
  input.type = 'text';
  input.placeholder = 'Escribe una nueva tarea...';
  input.required = true;
  input.disabled = Boolean(disabled);

  const button = document.createElement('button');
  button.type = 'submit';
  button.className = 'btn-add';
  button.textContent = 'Agregar Tarea';
  button.disabled = Boolean(disabled);

  form.append(input, button);

  form.addEventListener('submit', (e) => {
    e.preventDefault();
    const value = input.value.trim();
    if (!value) return;
    onAddTask(value); // Callback al componente padre (App)
    input.value = '';
  });

  return form;
}
```

3.4.3. Componente 3: TaskItem (js/components/TaskItem.js)

Recibe una tarea y el callback `onDeleteTask`:

```
export function TaskItem({ task, onDeleteTask }) {
  const li = document.createElement('li');
  li.className = `task-item ${task.completed ? 'completed' : ''}`;

  const span = document.createElement('span');
  span.className = 'task-title';
  span.textContent = task.title;

  const actions = document.createElement('div');
  actions.className = 'task-actions';

  const badge = document.createElement('span');
  badge.className = `badge ${task.completed ? 'badge-completed' : 'badge-pending'}`;
  badge.textContent = task.completed ? 'Completada' : 'Pendiente';
```

```
  const btnDelete = document.createElement('button');
  btnDelete.className = 'btn-delete';
  btnDelete.textContent = 'Eliminar';
  btnDelete.onclick = () => onDeleteTask(task.id); // Callback al padre

  actions.append(badge, btnDelete);
  li.append(span, actions);
  return li;
}
```

3.4.4. Componente 4: TaskList (js/components/TaskList.js)

Recorre el array de tareas mapeando cada elemento a un `TaskItem`:

```
import { TaskItem } from './TaskItem.js';

export function TaskList({ tasks, onDeleteTask }) {
  const ul = document.createElement('ul');
  ul.className = 'task-list';

  if (!tasks || tasks.length === 0) {
    const empty = document.createElement('li');
    empty.className = 'empty-message';
    empty.textContent = 'No hay tareas registradas en la API.';
    ul.appendChild(empty);
    return ul;
  }

  tasks.forEach((task) => {
    ul.appendChild(TaskItem({ task, onDeleteTask }));
  });

  return ul;
}
```

3.4.5. Componente 5 y 6: LoadingSpinner y ErrorAlert

```
// js/components/LoadingSpinner.js: Muestra el estado PENDING de la Promesa
export function LoadingSpinner({ message = 'Consultando API remota...' }) {
  const container = document.createElement('div');
  container.className = 'loading-container';

  const spinner = document.createElement('div');
  spinner.className = 'spinner';

  const span = document.createElement('span');
  span.textContent = message;

  container.append(spinner, span);
  return container;
}
```

```
// js/components/ErrorAlert.js: Muestra el estado REJECTED de la Promesa
export function ErrorAlert({ message, onRetry }) {
  const alert = document.createElement('div');
  alert.className = 'error-alert';

  const text = document.createElement('span');
```

```
  text.textContent = message;
  alert.appendChild(text);

  if (onRetry) {
    const btnRetry = document.createElement('button');
    btnRetry.className = 'btn-retry';
    btnRetry.textContent = 'Reintentar';
    btnRetry.onclick = onRetry;
    alert.appendChild(btnRetry);
  }

  return alert;
}
```

3.5 Fase 5: Componente Raíz y Orquestador de Estado (js/components/App.js) (25 Minutos)

El componente `App` administra el estado centralizado, reacciona a los eventos de los hijos y consume el servicio asíncrono controlando los estados de las Promesas:

```
// js/components/App.js: Componente Raiz con Estado y Orquestacion
import { getTasks, createTask, deleteTask } from '../services/taskService.js';
import { Header } from './Header.js';
import { TaskForm } from './TaskForm.js';
import { TaskList } from './TaskList.js';
import { LoadingSpinner } from './LoadingSpinner.js';
import { ErrorAlert } from './ErrorAlert.js';

export function App() {
  const container = document.createElement('main');
  container.className = 'app-container';

  // 1. Estado centralizado de la aplicacion
  const state = {
    tasks: [],
    loading: false,
    loadingMessage: '',
    error: null
  };

  // 2. Funcion de renderizado dinamico (construye el DOM segun el estado actual)
  function render() {
    container.innerHTML = ''; // Limpia el arbol anterior para redibujar

    // Encabezado
    container.appendChild(Header({
      title: 'TaskFlow Components',
      subtitle: 'Arquitectura de Componentes en Vanilla JS y Promesas (GET, POST, DELETE)'
    }));

    // Formulario de Creacion (POST)
    container.appendChild(TaskForm({
      disabled: state.loading,
      onAddTask: handleAddTask
    }));

    // Indicador de Carga (Estado PENDING)
    if (state.loading) {
      container.appendChild(LoadingSpinner({ message: state.loadingMessage }));
    }

    // Alerta de Error (Estado REJECTED)
```

```
  if (state.error) {
    container.appendChild(ErrorAlert({
      message: state.error,
      onRetry: handleLoadTasks
    }));
  }

  // Boton de refresco manual
  const refreshBtn = document.createElement('button');
  refreshBtn.className = 'btn-refresh';
  refreshBtn.textContent = 'Refrescar Lista desde API';
  refreshBtn.disabled = state.loading;
  refreshBtn.onclick = handleLoadTasks;
  container.appendChild(refreshBtn);

  // Lista de Tareas (Estado FULFILLED)
  container.appendChild(TaskList({
    tasks: state.tasks,
    onDeleteTask: handleDeleteTask
  }));
}

// 3. Manejadores Asincronos de Eventos (Consumo de Promesas)
async function handleLoadTasks() {
  state.loading = true;
  state.loadingMessage = 'Consultando API remota... (Estado: PENDING)';
  state.error = null;
  render();

  try {
    state.tasks = await getTasks(); // Promesa GET resuelta (FULFILLED)
  } catch (err) {
    state.error = `Error al consultar la API: ${err.message}. Verifica el servidor en el puerto 3001.`;
  } finally {
    state.loading = false;
    render();
  }
}

async function handleAddTask(title) {
  state.loading = true;
  state.loadingMessage = `Guardando "${title}" en la API... (Estado: PENDING)`;
  state.error = null;
  render();

  try {
    const nueva = await createTask(title); // Promesa POST resuelta (FULFILLED)
    state.tasks.push(nueva);
  } catch (err) {
    state.error = `Error al crear tarea: ${err.message}`;
  } finally {
    state.loading = false;
    render();
  }
}

async function handleDeleteTask(id) {
  state.loading = true;
  state.loadingMessage = `Eliminando tarea ID ${id}... (Estado: PENDING)`;
  state.error = null;
  render();
```

```
    try {
      await deleteTask(id); // Promesa DELETE resuelta (FULFILLED)
      state.tasks = state.tasks.filter((t) => String(t.id) !== String(id));
    } catch (err) {
      state.error = `Error al eliminar tarea: ${err.message}`;
    } finally {
      state.loading = false;
      render();
    }
  }

  // Carga inicial automatica de datos
  handleLoadTasks();

  return container;
}
```

3.6 Fase 6: Montaje en el Punto de Entrada (js/main.js) (5 Minutos)

En `js/main.js`, importamos `App` y la insertamos en el contenedor raíz `#root`:

```
// js/main.js: Punto de entrada de la aplicacion
import { App } from './components/App.js';

const root = document.querySelector('#root');
root.appendChild(App());
```

3.7 Fase 7: Ejecución y Experimentación en Vivo (15 Minutos)

Inicie el servidor estático del frontend en `http://localhost:3000`:

```
npx live-server --port=3000
```

Realice los siguientes **4 experimentos guiados en vivo**:

1. **Experimento 1: Inspección del DOM Vacío vs. Renderizado Dinámico**
   - En su navegador, abra DevTools (`F12`) y seleccione la pestaña **Elements (Elementos)**.
   - Abra el archivo fuente original `index.html` y compruebe que solo contiene `<div id="root"></div>`.
   - Observe en el árbol del inspector cómo JavaScript construyó toda la jerarquía de elementos (`header`, `form`, `button`, `ul`, `li`) dentro de `#root`.
2. **Experimento 2: Ciclo de Vida de Promesa en Creación (POST)**
   - Escriba una tarea en el input del componente `TaskForm` y haga clic en "Agregar Tarea".
   - Observe cómo el componente `App` cambia inmediatamente su estado a `loading: true`, renderizando el `LoadingSpinner` (**Estado Pending**).
   - Al resolver la Promesa, la API entrega el ID asignado, el componente añade la tarea a `state.tasks` y re-renderiza la lista (**Estado Fulfilled**).
3. **Experimento 3: Flujo de Event Callbacks en Eliminación (DELETE)**
   - Haga clic en el botón "Eliminar" de un elemento de la lista.
   - Observe cómo el componente hijo `TaskItem` notifica el evento al padre `App` mediante `onDeleteTask(id)`.
   - Al completarse la Promesa `DELETE`, el item desaparece de la vista sin recargar la página.

4. **Experimento 4: Estado REJECTED y Resiliencia con ErrorAlert**
   - En la terminal de Express, detenga el servidor con `Ctrl + C`.
   - En el navegador, intente agregar una tarea o presione el botón de refresco.
   - Compruebe que la Promesa de red falla, la ejecución salta al bloque `catch` y el componente `ErrorAlert` se dibuja con el botón de "Reintentar".
   - Reinicie el servidor con `npm start`, presione "Reintentar" y verifique la recuperación transparente de la aplicación.

4 Cuestionario de Análisis Conceptual y Discusión Técnica

Responda en pareja las siguientes preguntas conceptuales:

1. **Arquitectura de Componentes frente a HTML tradicional:** ¿Cuáles son las ventajas de tener un archivo `index.html` que solo contenga `<div id="root"></div>` y generar la interfaz mediante componentes funcionales en JavaScript? ¿Cómo facilita esto la modularidad, depuración y mantenimiento del código frente a un único script monolítico?
2. **Flujo Unidireccional de Datos (Props y Callbacks):** Explique cómo viajan los datos y las acciones entre el componente `App`, `TaskList` y `TaskItem`:
   - ¿Cómo recibe `TaskItem` la información de la tarea que debe pintar?
   - ¿Cómo le avisa `TaskItem` al componente `App` que el usuario presionó el botón de eliminar?
3. **Separación de Responsabilidades (Service Layer vs. Componentes):** ¿Por qué es una buena práctica de ingeniería extraer las funciones `fetch` al archivo `taskService.js` en lugar de escribir `fetch()` directamente dentro de cada componente visual?
4. **Manejo de Estados Asíncronos en la UI:** Describa qué componentes visuales se montan o desmontan del DOM según los tres estados de la Promesa:
   - En estado **Pending**:
   - En estado **Fulfilled**:
   - En estado **Rejected**:

5 Rúbrica de Autoevaluación Formativa Guiada (Escala 0.0 a 5.0)

6 Instrucciones de Verificación y Cierre en Aula

Al finalizar la sesión práctica, presente ante el profesor o monitor:

1. Inspección en DevTools mostrando que el archivo `index.html` solo tiene `<div id="root"></div>` y que toda la UI fue construida mediante componentes JavaScript.
2. Demostración de creación de tarea (`POST`) y eliminación (`DELETE`) con actualización dinámica de la interfaz.
3. Demostración de resiliencia: apagar la API y mostrar la aparición del componente `ErrorAlert` con la opción de reintentar.

Criterio Evaluado	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)	Puntaje
1. Arquitectura de Componentes y DOM	Estructura la aplicación en componentes funcionales modulares a partir de un único `<div id="root"></div>`, manipulando el DOM con `createElement` y `textContent`.	Implementa componentes pero recurre a estructuras HTML estáticas fuera de `#root` o abusa de `innerHTML`.	No modulariza en componentes o mantiene toda la vista codificada rígidamente en el archivo HTML.	**1.5 pts**
2. Consumo de Promesas (Service Layer)	Implementa el servicio `taskService.js` con `fetch` y `async/await` para `GET`, `POST` y `DELETE`, gestionando `try/catch/finally`.	Consume las Promesas pero escribe las llamadas directamente en los componentes sin capa de servicio.	Errores de sintaxis en el consumo de red o incapacidad de procesar las respuestas de la API.	**1.5 pts**
3. Manejo de Estado y Eventos (Props / Callbacks)	El componente `App` centraliza el estado y orquesta el renderizado dinámico, pasando props y manejando callbacks correctamente.	Maneja el estado pero la actualización visual del DOM presenta inconsistencias al crear o eliminar elementos.	La interfaz no responde a eventos o colapsa ante cambios en los datos.	**1.0 pt**
4. Discusión Conceptual y Resiliencia	Explica con precisión el flujo unidireccional de datos y demuestra la captura de errores en `ErrorAlert` al apagar la API.	Comprende la estructura de componentes pero tiene dudas sobre la relación entre props y callbacks.	No responde el cuestionario o desconoce el funcionamiento asíncrono de las Promesas.	**1.0 pt**
			PUNTAJE FORMATIVO TOTAL:	**5.0 pts**